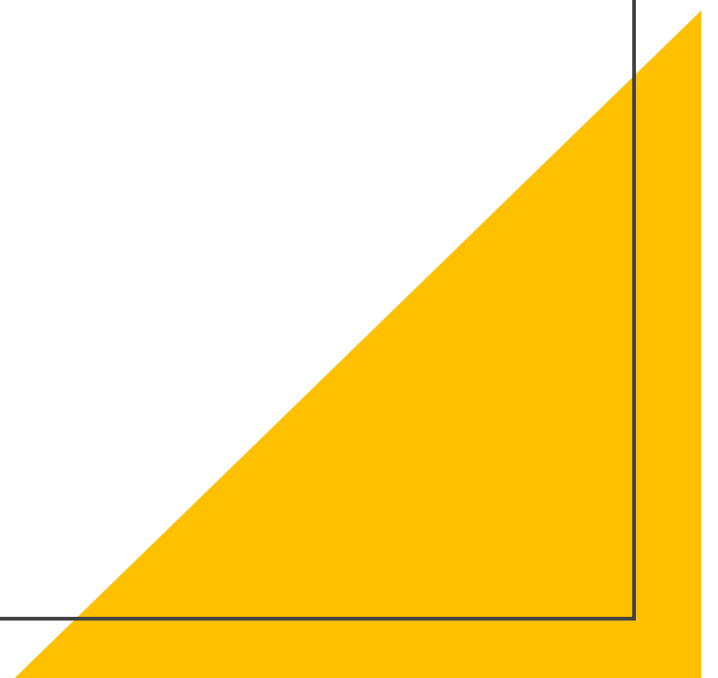


Signals



Signals是什麼？

signals跟訂閱其實有點相像，都是需要去監測一個值的變化然後觸發資料的修改或者呈現。

不過signals是後面Angular v16之後推出的新功能，主要是用來讓你處理不需要"等待"的資料變化與呈現，所以如果你現在是需要呼叫Api這種要"等待"後端回應的資料就還是需要使用到原本的訂閱寫法。



Signal

Signals怎麼寫？

還記得我們先前寫過的loading.service.ts的內容嗎？

我們特別宣告了兩個變數，一個變數用來可被觀察者(loading\$)，一個變數單純的開放給其他頁面觀察他的值的變化(_loading\$)，並且我們在下面新增了兩個方法用來改變可被觀察者的內容。

```
export class LoadingService {  
  private loading$ = new Subject<boolean>();  
  _loading$ = this.loading$.asObservable();  
  
  constructor() { }  
  
  show() {  
    this.loading$.next(true);  
  }  
  
  hide() {  
    this.loading$.next(false);  
  }  
}
```

loading.service.ts

Signals怎麼寫?

緊接著我們要在要訂閱該內容的頁面中去訂閱_loading\$，並且等待他發生變化觸發程式。

```
export class AppComponent {  
  
  constructor(private loadingService: LoadingService) {}  
  
  ngOnInit(): void {  
    this.loadingService._loading$  
      .subscribe((res) => {  
        console.log(res);  
      });  
  }  
}
```

app.component.ts

Signals怎麼寫?

那如果我現在要改成寫signals，第一步我們先來到loading.service.ts中把原有的可被觀察者的值修改成的signals。

並且把下面兩個方法的寫法改成.set()，這樣我們就修改完畢了，不過signals必須一定要給初始值所以就先設定一個預設false。

```
export class LoadingService {  
  // private loading$ = new Subject<boolean>();  
  // _loading$ = this.loading$.asObservable();  
  loading = signal<boolean>(false);  
  
  constructor() { }  
  
  show() {  
    // this.loading$.next(true);  
    this.loading.set(true);  
  }  
  
  hide() {  
    // this.loading$.next(false);  
    this.loading.set(false);  
  }  
}
```

loading.service.ts

Signals怎麼寫?

緊接著來到原本呼叫的地方把對應的內容修改成底下的樣子，因為signal是一個函式所以你如果要查看裡面的值需要寫成 `console.log(this.loading());`。

```
export class AppComponent {  
  
  loading!: Signal<boolean>;  
  constructor(private loadingService: LoadingService) {}  
  
  ngOnInit(): void {  
    // this.loadingService._loading$  
    //   .subscribe((res) => {  
    //     console.log(res);  
    //   });  
    this.loading = this.loadingService.loading;  
  }  
}
```

app.component.ts

Signals怎麼寫？

這個時候如果你要把這個值應用在HTML上，我們就可以用內嵌繫節將這個函式應用在HTML上去做應用，當這個signals的值做變更的時候畫面的呈現也會及時做改變。

```
@if (loading()) {  
  <h5>loading的值: {{ loading() }}</h5>  
}
```

app.component.html

偵測Signals

這時候如果我們需要在loading的值變更的時候做一些資料處理，他也有給你偵測他的內容變更的時候的觸發方法，不過這段程式寫的地方要特別注意，你要寫在建構式(constructor)中，寫法是固定的所以可以的話就記起來。

```
export class AppComponent {  
  
  loading!: Signal<boolean>;  
  constructor(private loadingService: LoadingService) {  
    effect(() => console.log(this.loading()));  
  }  
  
  ngOnInit(): void {  
    this.loading = this.loadingService.loading;  
    setTimeout(() => {  
      this.loadingService.show();  
    }, 1000);  
  }  
}
```

app.component.ts

注意事項

不過要注意signals適用在自己程式裡面的值的變化跟偵測，如果我現在是要呼叫Api這種需要等待回應的寫法就還是要去使用訂閱這種寫法去等待Api回應並且做資料處理。

结束

